

Program 1

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

# Step 1: Load the California Housing dataset
data = fetch_california_housing(as_frame=True)
housing_df = data.frame
housing_df.hist()

# Step 2: Create histograms for numerical features
numerical_features =
housing_df.select_dtypes(include=[np.number]).columns

# Plot histograms
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.histplot(housing_df[feature], kde=True, bins=30, color='blue')
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()

# Step 3: Generate box plots for numerical features
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3, 3, i + 1)
    sns.boxplot(x=housing_df[feature], color='orange')
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()

# Step 4: Identify outliers using the IQR method
print("Outliers Detection:")
outliers_summary = {}
for feature in numerical_features:
    Q1 = housing_df[feature].quantile(0.25)
    Q3 = housing_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = housing_df[(housing_df[feature]<lower_bound)|
(housing_df[feature]>upper_bound)]
    outliers_summary[feature] = len(outliers)
    print(f"{feature}: {len(outliers)} outliers")

# Optional: Print a summary of the dataset
```

```
print("\nDataset Summary:")
print(housing_df.describe())
outliers
```

Program 2

```
import pandas as pd
import seaborn as sn
import matplotlib as plt
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame
matrix = df.corr()
print(matrix)
sn.heatmap(matrix, cmap = "coolwarm", annot=True)
print(housing.feature_names)
sn.pairplot(df, vars=["AveRooms", "AveBedrms"])
sn.pairplot(df, vars=["Longitude", "Latitude"])
sn.pairplot(df, vars=["Longitude", "AveOccup"])
sn.pairplot(df, vars=["MedInc", "AveRooms"])
sn.pairplot(df)
```

Program 3

```
import numpy as np
import pandas as pd
import seaborn as sn
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

iris_bunch = load_iris(as_frame=True)
iris_df = iris_bunch.frame
iris_df = iris_df.drop('target', axis=1) #axis= 1 for column, 0 for row
print(iris_df.head(5)) #output

iris_scaled = StandardScaler().fit_transform(iris_df)

pca = PCA(n_components=2)
iris_pca = pca.fit_transform(iris_scaled)

explained_variance = pca.explained_variance_ratio_
print(f"Explained variance ratio: {explained_variance}")

pca_df = pd.DataFrame(data = iris_pca, columns = ['PC1', 'PC2'])
final_df = pd.concat([pca_df, iris_bunch.target], axis = 1)
final_df["target"] = iris_bunch.target_names[iris_bunch.target]
print(final_df.head(5))#output

sn.pairplot(final_df, hue= "target", diag_kind='kde')
```

Program 4

```
import numpy as np
import pandas as pd

data = pd.read_csv('data.csv')
print('Given data:')
print(data)

data_arr = np.array(data)[:, :-1]
features = data_arr.shape[1]

sp_h = ['0']*features
print(f'Initial sp_h:{sp_h}')

target = np.array(data)[:,-1]

for i in range(len(target)):
    if target[i] == 'yes':
        for j in range(0,features):
            if sp_h[j] == '0':
                sp_h[j] = data_arr[i][j]
            elif sp_h[j] != data_arr[i][j]:
                sp_h[j] = '?'
            else:
                pass
        print(f'for step {i}:{sp_h}')

print(f'Final sp_h:{sp_h}')
```